

SEARCHING AND SORTING

Insertion Sort - Part 1

```
#include <stdio.h>

void print(int ar_size, int* ar) {
    int i;
    for(i=0; i<ar_size; i++) {
        printf("%d ", ar[i]);
    }
    printf("\n");
}
```

```
#include <string.h>
#include <math.h>
#include <stdlib.h>
#include <assert.h>
```

```
/* Head ends here */
```

```
void insertionSort(int ar_size, int * ar) {
    int j = ar_size-1;
    int v = ar[j];
    while(v < ar[j-1]) {
        ar[j] = ar[j-1];
        j--;
    }
    print(ar_size, ar);
    ar[j] = v;
```

```
    print(ar_size, ar);
}

/* Tail starts here */
int main() {

    int _ar_size;
    scanf("%d", &_ar_size);
    int _ar[_ar_size], _ar_i;
    for(_ar_i = 0; _ar_i < _ar_size; _ar_i++) {
        scanf("%d", &_ar[_ar_i]);
    }

    insertionSort(_ar_size, _ar);

    return 0;
}
```

Insertion Sort - Part 2

```
#include <stdio.h>

#include <string.h>

#include <math.h>

#include <stdlib.h>

#include <assert.h>

/* Head ends here */

void insertionSort(int ar_size, int * ar) {
    for (int i = 1; i < ar_size; ++i) {
        int j = i - 1;
        int p = ar[i];
        while (j >= 0 && p < ar[j]) {
            ar[j+1] = ar[j];
            j--;
        }
        ar[j+1] = p;
        printf("%d", ar[0]);
        for (int k = 1; k < ar_size; ++k) {
            printf(" %d", ar[k]);
        }
        printf("\n");
    }
}

/* Tail starts here */

int main() {
    int _ar_size;
    scanf("%d", &_ar_size);
```

```
int _ar[_ar_size], _ar_i;  
for(_ar_i = 0; _ar_i < _ar_size; _ar_i++) {  
    scanf("%d", &_ar[_ar_i]);  
}  
  
insertionSort(_ar_size, _ar);  
  
    return 0;  
}
```

Correctness and the Loop Invariant

```
#include <stdio.h>
#include <string.h>
#include <math.h>
#include <stdlib.h>
#include <assert.h>
/* Head ends here */
#include <stddef.h>
void insertionSort(int ar_size, int * ar) {
    int i,j;
    int value;
    for(i=1;i<ar_size;i++)
    {
        value=ar[i];
        j=i-1;
        while(j>=0 && value<ar[j])
        {
            ar[j+1]=ar[j];
            j=j-1;
        }
        ar[j+1]=value;
    }
    for(j=0;j<ar_size;j++)
    {
        printf("%d",ar[j]);
        printf(" ");
    }
```

```
}  
/* Tail starts here */  
int main(void) {  
    int _ar_size;  
    scanf("%d", &_ar_size);  
    int _ar[_ar_size], _ar_i;  
    for(_ar_i = 0; _ar_i < _ar_size; _ar_i++) {  
        scanf("%d", &_ar[_ar_i]);  
    }  
  
    insertionSort(_ar_size, _ar);  
  
    return 0;  
}
```

Running Time of Algorithms

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#include <math.h>
```

```
#include <stdlib.h>
```

```
#include <assert.h>
```

```
/* Head ends here */
```

```
void insertionSort(int ar_size, int * ar,int *shifts) {
```

```
    int temp=ar[ar_size-1],i;
```

```
    for(i=ar_size-2;i>=0;i--)
```

```
    {
```

```
        if(ar[i]>temp){
```

```
            ar[i+1]=ar[i];
```

```
            *shifts=*shifts+1;
```

```
        }
```

```
    else
```

```
        break;
```

```
    }
```

```
    ar[i+1]=temp;
```

```
}
```

```
/* Tail starts here */
```

```

int main() {

    int _ar_size,i,j,shifts=0;
    scanf("%d", &_ar_size);
    int _ar[_ar_size], _ar_i;
    for(_ar_i = 0; _ar_i < _ar_size; _ar_i++) {
        scanf("%d", &_ar[_ar_i]);
    }
    for(i=2;i<=_ar_size;i++)
    {
        insertionSort(i, _ar,&shifts);
    }
    printf("%d",shifts);
    return 0;
}

```

Counting Sort 1

```

#include <stdio.h>
#include <string.h>
#include <math.h>
#include <stdlib.h>

```

```

int main() {

```

```

    int n,i;
    int b[100],a;

```



```
scanf("%d",&n);
```

```
for(i=0;i<100;i++)
```

```
{
```

```
    b[i]=0;
```

```
}
```

```
for(i=0;i<n;i++)
```

```
{
```

```
    scanf("%d",&a);
```

```
    b[a]++;
```

```
}
```

```
for(i=0;i<100;i++)
```

```
{
```

```
    printf("%d ", b[i]);
```

```
}
```

```
return 0;
```

```
}
```

RECURSION AND BIT MANIPULATION

Crossword Puzzle

The Power Sum

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#include <math.h>
```

```
#include <stdlib.h>
```

```
int the_power_sum(int n, int m,int p){
```

```
    int tmp = pow(m,p);
```

```
    if(tmp == n) return 1;
```

```
    if(tmp > n) return 0;
```

```
    return the_power_sum(n,m+1,p) + the_power_sum(n-tmp, m+1,p);
```

```
}
```

```
int main() {
```

```
    int n,p;
```

```
    scanf("%d%d",&n,&p);
```

```
    printf("%d", the_power_sum(n,1,p));
```

```
    return 0;
```

```
}
```

Counter Game

```
#include <stdio.h>
#include <string.h>
#include <math.h>
#include <stdlib.h>

int isPow2(long unsigned int);
unsigned long int largePow(long unsigned int);
int main() {

    int t,i,win;
    long unsigned int n;
    scanf("%d",&t);
    for(i=0;i<t;++i)
    {
        win=0;
        scanf("%lu",&n);
        if(n==1)
            printf("Richard\n");
        else
        {
            while(n!=1)
            {
                if(isPow2(n))
                    n>>=1;
                else
                    n-=largePow(n);
                ++win;
            }
        }
    }
}
```

```

    }
}

    if(win%2==0)
        printf("Richard\n");
    else
        printf("Louise\n");
}

return 0;
}

int isPow2(long unsigned int n)
{
    return !(n&(n-1));
}

long unsigned int largePow(long unsigned int n)
{
    long unsigned int m;
    while(n)
    {
        m=n;
        n=n&(n-1);
    }
    return m;
}

```

GREEDY AND DYNAMIC PROGRAMMING

The Coin Change Problem

Sherlock and Cost

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>

int main() {
    int T,N,B,L,R,ML,MR,X,Y,P,Q;
    scanf("%d",&T);
    for(int i = 0; i < T; i++) {
        scanf("%d",&N);
        for(int j = 0; j < N; j++) {
            scanf("%d",&B);
            if(j) {
                X = L - 1 + ML;
                Y = R - 1 + MR;
                P = abs(L - B) + ML;
                Q = abs(R - B) + MR;
                ML = (X > Y ? X : Y);
                MR = (P > Q ? P : Q);
            } else {
                ML = MR = 0;
            }
            L = 1;
            R = B;
        }
        printf("%d\n", (ML > MR ? ML : MR));
    }
    return 0;
}
```

Marc's Cakewalk

```
#include <math.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <assert.h>
#include <limits.h>
#include <stdbool.h>
void swap(int *a,int *b)
{
    int temp;
    temp = *a;
    *a = *b;
    *b=temp;
}
int partition(int *x,int start,int end)
{
    int pivot,pindex,i;
    pivot = x[end];
    pindex = start;
    for(i=start;i<end;i++)
    {
        if(x[i]>=pivot)
        {
            swap(&x[i],&x[pindex]);
            pindex = pindex + 1;
        }
    }
}
```

```

    }
    swap(&x[pindex],&x[end]);
    return pindex;
}

void quicksort(int *x,int start,int end)
{
    if(start<end)
    {
        int i = partition(x,start,end);
        quicksort(x,start,i-1);
        quicksort(x,i+1,end);
    }
}

int main(){
    int n, calories_i, *calories;
    int i;
    int sum = 0;
    scanf("%d",&n);
    calories = malloc(sizeof(int) * n);
    for(calories_i = 0; calories_i < n; calories_i++)
    {
        scanf("%d",&calories[calories_i]);
    }
    quicksort(calories,0,n-1);

    for(i=0;i<n;i++)
    {
        sum += calories[i]*((int)pow(2,i));
    }
}

```

```
}
```

```
printf("%d",sum);
```

```
// your code goes here
```

```
return 0;
```

```
}
```


STRINGS

Strong Password

```
#include <math.h>
```

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#include <stdlib.h>
```

```
#include <assert.h>
```

```
#include <limits.h>
```

```
#include <stdbool.h>
```

```
int minimumNumber(int n, char* password) {
```

```
    int L_c = 0;
```

```
    int U_c = 0;
```

```
    int no = 0;
```

```
    int s_c = 0;
```

```
    int len = strlen(password);
```

```
    int i;
```

```
    for(i = 0; i < n; i++)
```

```
    {
```

```
        if(password[i] >= 'a' && password[i] <= 'z')
```

```
            L_c++;
```

```
        else if(password[i] >= 'A' && password[i] <= 'Z')
```

```
            U_c++;
```

```
        else if(password[i] >= '0' && password[i] <= '9')
```

```
        no++;  
    else  
        s_c++;  
}
```

```
int add = 0;  
if(L_c == 0)  
    add++;  
if(U_c == 0)  
    add++;  
if(no == 0)  
    add++;  
if(s_c == 0)  
    add++;
```

```
len = len + add;
```

```
if(len < 6)  
    add = add + 6 - len;
```

```
return add;  
}
```

```
int main() {  
    int n;  
    scanf("%i", &n);  
    char* password = (char *)malloc(512000 * sizeof(char));  
    scanf("%s", password);
```

```
int answer = minimumNumber(n, password);  
printf("%d\n", answer);  
return 0;  
}
```

Caesar Cipher

```
#include <stdio.h>  
#include <string.h>  
#include <math.h>  
#include <stdlib.h>
```

```
int main() {
```

```
    int n,i,j,k;
```

```
    char ar[101];
```

```
    unsigned char x;
```

```
    scanf("%d",&n);
```

```
    scanf("%s",ar);
```

```
    scanf("%d",&k);
```

```
    for(i=0;i<n;i++)
```

```
    {
```

```
        x=ar[i];
```

```
        if(x>=97 && x<=122)
```

```
        {
```

```
            x=x+(k%26);
```

```
    if(x>122)
    {
        x=96+(x-122);
    }
    ar[i]=x;
}
else if(x>=65 && x<=90)
{
    x=x+(k%26);
    if(x>90)
    {
        x=64+(x-90);
    }
    ar[i]=x;
}
}
printf("%s",ar);
return 0;
}
```

Pangrams

```
#include <stdio.h>

char s[10000];

int main()
{
    gets(s);
    int f[300]={0},ans=0,i;
    int l=strlen(s);
    for(i=0;i<l;i++)
    {
        if(s[i]==' ')
            continue;
        if(s[i]>='a')
            s[i]=s[i]-('a'-'A');
        if(!(f[s[i]]++))
            ans++;
    }
    if(ans!=26)
        printf("not ");
    printf("pangram\n");
    return 0;
}
```

RANGE QUERIES

Prefix Sum Array

Populate a Prefix-Sum Array from an array of integer elements.

Also create a function to calculate the sum of elements between a given range using the Prefix-Sum Array.

```
#include<stdio.h>
```

```
void display(int arr[], int n)
```

```
{
    int i;
    for (i=0;i<n;i++)
    {
        printf("\t %d ",arr[i]);
    }
}
```

```
create_prefix_sum_array(int arr[], int n)
```

```
{
    int i;
    for (i = 1; i<n; i++)
    {
        arr[i] = arr[i] + arr[i-1];
    }
}
```

```
void sum(int arr[], int n, int a, int b){
```

```
    int result;
```

```
    if (a == 0)
        result = arr[b];
    else
        result = arr[b] - arr[a-1];
    printf("The sum is %d", result);
}
```

```
int main(){
    int n, i, arr[10], a, b;
    printf("enter the number of elements\n");
    scanf("%d",&n);
    printf("enter the array elements\n");
    for (i=0;i<n;i++)
    {
        scanf("%d",&arr[i]);
    }

    printf("\n Original array");
    display(arr, n);

    create_prefix_sum_array(arr, n);

    printf("\n Prefix sum array");
    display(arr,n);

    printf("\n enter the range to find the sum");
    scanf("%d%d",&a,&b);
    sum(arr, n, a, b);
}
```

```

        return 0;
    }

```

Fenwick Tree Construction

Populate a Fenwick Tree from an array of integer elements.

Given the value of a node in a Fenwick Tree

$$\text{tree}[k] = \text{sum}_q(k - p(k) + 1, k)$$

Where, $p(k) = k \& -k$ and denotes the largest power of two that divides k

```

#include<stdio.h>

```

```

#include<math.h>

```

```

int sum(int arr[], int n, int a, int b){

```

```

    int i, sum=0;

```

```

    for (i = a; i<=b; i++)

```

```

    {

```

```

        sum += arr[i] ;

```

```

    }

```

```

    return sum;

```

```

}

```

```

void create_fenwick_tree(int T[], int arr[], int n)

```

```

{

```

```

    int a, b, k;

```

```

    for (k=1; k<=n; k++)

```

```

    {

```

```

        a = k - (k&-k) + 1;

```



```
        b = k;  
        T[k] = sum(arr,n,a,b);  
    }
```

```
}
```

```
void display(int arr[], int n)  
{  
    int i;  
    for (i=1;i<=n;i++)  
    {  
        printf("\t %d ",arr[i]);  
    }  
}
```

```
int main(){  
    int n, i, arr[100], T[100] , a, b;  
  
    // printf("the power of 6 is %u \n", largestPowerOf2(7)) ;  
    printf("enter the number of elements\n");  
    scanf("%d",&n);  
    printf("enter the array elements\n");  
    for (i=1;i<=n;i++)  
    {  
        scanf("%d",&arr[i]);  
    }  
    printf("\n Original array");
```

```

display(arr, n);
create_fenwick_tree(T, arr, n);
printf("\n Fenwick Tree");
display(T,n);
return 0;
}

```

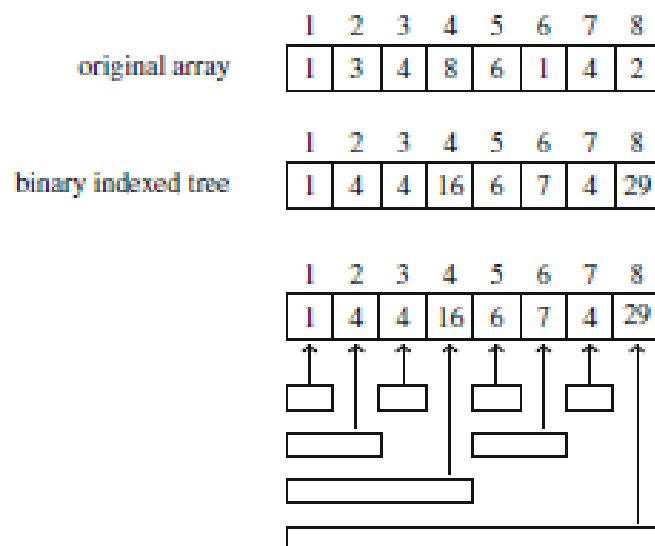
Fenwick Tree - Sum of Elements

Given a Fenwick Tree (Binary indexed Tree) with the value of at node k calculated as

$$\text{tree}[k] = \text{sum}_q (k - p(k) + 1, k)$$

Where, $p(k) = k \& -k$ and denotes the largest power of two that divides k.

Create a function to calculate sum of first 'k' elements in the Fenwick Tree.



```
#include<stdio.h>
```

```
#include<math.h>
```

```
int sum(int T[], int k) {
```

```
    int s = 0;
```

```
    while (k >= 1) {
```

```
        s += T[k];
```

```
        k -= k&-k;
```

```
    }
```

```
    printf("\n The sum is %d",s) ;
```

```
}
```

```
void display(int arr[], int n)
```

```
{
```

```
    int i;
```

```
    for (i=1;i<=n;i++)
```

```
    {
```

```
        printf("\t %d ",arr[i]);
```

```
    }
```

```
}
```

```
int main(){
```

```
    int n, k, T[100] = {0, 1, 4, 4, 16, 6, 7, 4, 29};
```

```
    printf("\n Fenwick Tree");
```

```
    display(T,n);
```

```
    printf("\n Enter the value for k");
```

```
    scanf("%d",&k);
```

```
    sum(T, k);
```

```
    return 0;
```

```
}
```